

TypeScript

Interview Questions & Answers

100 most-asked questions · Basic → Advanced · with code & memory hooks

Prepared for **Suraj Khonde** · Full-Stack Engineer

How to use this: Each answer leads with a short, plain-English idea you can repeat in an interview, followed by a tiny code example. The bold *memory hooks* are one-liners worth memorising. Skim Level 1 to warm up, then spend most of your time on Levels 2–3 where interviewers dig deepest. Good luck!

LEVEL 1 — Fundamentals & The Type System

The base layer every interviewer checks first. Know these cold.

Q1. What is TypeScript and how does it relate to JavaScript?

TypeScript is a **superset of JavaScript** that adds an optional, **static** type system. Any valid JS is valid TS. It compiles ("transpiles") down to plain JavaScript, so the browser/Node never sees the types.

Memory hook: *"JS + types, checked at build time, erased at runtime."*

Q2. What are the main benefits of using TypeScript?

Catch bugs at **compile time** instead of in production, **self-documenting** code, world-class **autocomplete/refactoring** in editors, and safer large-team codebases. The cost is a build step and some annotation effort.

Q3. What are the basic primitive types?

string, **number**, **boolean**, **bigint**, **symbol**, plus **null** and **undefined**. There is **no int/float** — all numbers are **number**. Note the lowercase: use **string**, not **String** (the wrapper object type).

```
let name: string = 'Suraj';
let age: number = 30;
let active: boolean = true;
let big: bigint = 100n;
```

Q4. Difference between **any**, **unknown**, and **never**?

any = turn checking **off** (anything goes — avoid it). **unknown** = type-safe **any**; you must **narrow** it before use. **never** = a value that **can't exist** (e.g. a function that always throws, or an impossible branch). **Hook:** **any = no check, unknown = check first, never = nothing.**

```
let a: any = 5;      a.foo();           // allowed (unsafe)
let u: unknown = 5; // u.foo()          // error until narrowed
if (typeof u === 'number') u.toFixed(); // ok
function fail(): never { throw new Error(); }
```

Q5. What is type inference?

TypeScript automatically figures out a type when you don't annotate it. **let x = 3** is inferred as **number**.

Best practice: let TS infer locals; annotate function **parameters**, **return types**, and **public APIs** explicitly.

```
let count = 5;           // inferred: number
const tag = 'hi';        // inferred: 'hi' (literal, because const)
```

Q6. **interface** vs **type alias** — when to use which?

Both describe object shapes. **interface** can be **re-opened/merged** (declaration merging) and is idiomatic for object/class contracts. **type** is more flexible — it can alias **unions, primitives, tuples, mapped/conditional types**. **Rule of thumb**: interfaces for objects you might extend; types for everything else.

```
interface User { id: number; name: string; }
type ID = string | number;           // type can do unions
type Point = { x: number; y: number };
```

Q7. What are union types?

A value that can be **one of several** types, written with **|**. You can only use members common to all members until you narrow it. **Hook**: ***union = OR.***

```
function pad(v: string | number) {
  return typeof v === 'number' ? ' '.repeat(v) : v;
}
```

Q8. What are intersection types?

Combine multiple types into one that has **all** their members, written with **&**. **Hook**: ***intersection = AND.*** Great for mixing in extra props.

```
type A = { id: number };
type B = { name: string };
type AB = A & B;    // { id: number; name: string }
```

Q9. What are literal types?

Types that allow **one exact value**, like **'GET'** or **42**. Combined with unions they model finite sets — far safer than a loose **string**.

```
type Method = 'GET' | 'POST' | 'PUT';
let m: Method = 'GET';    // only these 3 allowed
```

Q10. What does the **?** optional property mean?

It marks a property/parameter as possibly **absent** — its type becomes **T | undefined**. **name?: string** means the key may not be there at all.

```
interface Cfg { url: string; retries?: number; }
const c: Cfg = { url: '/api' };    // retries omitted = ok
```

Q11. What does **readonly** do?

Prevents reassigning a property **after creation** — compile-time only, not frozen at runtime. Use **ReadonlyArray<T>** / **readonly T[]** for immutable arrays.

```
interface P { readonly id: number; }
const p: P = { id: 1 };
// p.id = 2;    // error
const arr: readonly number[] = [1, 2];    // arr.push() errors
```

Q12. `T[]` vs `Array<T>` for arrays?

Identical — `string[]` and `Array<string>` mean the same thing. `T[]` is the common shorthand; `Array<T>` reads better with complex element types.

```
let xs: number[] = [1, 2, 3];
let ys: Array<{ id: number }> = [{ id: 1 }];
```

Q13. What are tuples?

A **fixed-length** array where each position has its own type. Order matters. Supports optional and rest elements, and labels for readability.

```
let pair: [string, number] = ['age', 30];
type RGB = [r: number, g: number, b: number];
type Args = [string, ...number[]]; // rest element
```

Q14. Explain enums (numeric, string, const).

enum defines a named set of constants. **Numeric** enums auto-increment from 0 and create a reverse map. **String** enums are clearer in logs. **const enum** is fully inlined and leaves no runtime object. Many teams prefer a union of string literals or **as const** instead.

```
enum Color { Red, Green, Blue } // 0,1,2
enum Dir { Up = 'UP', Down = 'DOWN' } // string enum
const enum Size { S, M, L } // inlined, erased
```

Q15. `void` vs `undefined`?

void = a function returns **nothing meaningful** (the return value should be ignored). **undefined** = the actual value **undefined**. A **void**-typed callback may still return a value that's discarded — that's intentional and useful for **forEach**-style APIs.

```
function log(msg: string): void { console.log(msg); }
let u: undefined = undefined;
```

Q16. What does `strictNullChecks` change about `null/undefined`?

With it **on** (recommended), **null/undefined** are **not** assignable to other types unless you add them to a union. This forces you to handle the empty case and is the single biggest bug-preventer in TS.

```
let s: string = null; // error under strictNullChecks
let s2: string | null = null; // ok
```

Q17. How do you type a function?

Annotate each parameter and the return type. A function **type** (call signature) can be written inline for callbacks/variables.

```
function add(a: number, b: number): number { return a + b; }
const fn: (a: number, b: number) => number = add;
type Handler = (e: Event) => void;
```

Q18. Optional and default parameters?

`p?: T` makes a parameter optional. A default value (`p = 10`) also makes it optional and infers its type. Optional params must come **after** required ones.

```
function greet(name: string, title = 'friend', loud?: boolean) {}  
greet('Suraj');
```

Q19. Rest parameters?

Collect a variable number of trailing arguments into a typed array with `...`.

```
function sum(...nums: number[]): number {  
  return nums.reduce((a, b) => a + b, 0);  
}
```

Q20. `object` vs `{}` vs `Object`?

`object` = any non-primitive (arrays, functions, objects). `{}` = anything except `null/undefined` (almost useless, very loose). `Object` = the JS wrapper, basically `{}` — avoid. **Prefer a precise shape** like `{ id: number }` or `Record<...>`.

Q21. Type assertions (`as`) — what and the risk?

You tell the compiler "trust me, this is type X" with `value as X` (or `<X>value`, which clashes with JSX). It does **no runtime conversion** and can lie — overuse hides real bugs. Only assert when you genuinely know more than TS.

```
const el = document.getElementById('app') as HTMLDivElement;  
const n = '42' as unknown as number; // double-assert escape hatch
```

Q22. What is the non-null assertion `!`?

`x!` tells TS "this is definitely not null/undefined here." It removes `null/undefined` from the type **without** a runtime check — so a wrong assertion crashes at runtime. Prefer a real check when possible.

```
function f(el: HTMLElement | null) { el!.focus(); }
```

Q23. What is structural typing?

TS compares types by their **shape**, not their name ("duck typing"). If an object has all the required members, it's assignable — even if it was never declared to implement that type. This is core to how TS feels.

```
interface Named { name: string }  
const dog = { name: 'Rex', legs: 4 };  
const n: Named = dog; // ok — shape matches
```

Q24. What does `as const` (const assertion) do?

Freezes a literal to its **narrowest, readonly** form: strings become literal types, arrays become readonly tuples, and all props become **readonly**. Perfect for config objects and deriving types from values.

```
const ROLES = ['admin', 'user'] as const;  
type Role = typeof ROLES[number]; // 'admin' | 'user'
```

Q25. Why prefer **unknown** over **any** for external data?

any silently disables type-checking and spreads through your code. **unknown** keeps you safe — you **must** validate/narrow before using it, which is exactly what you want for API responses, **JSON.parse**, and **catch** errors.

```
const data: unknown = JSON.parse(raw);
if (typeof data === 'object' && data && 'id' in data) { /* safe */ }
```

Q26. **number** vs **Number**, **string** vs **String**?

Lowercase (**number**, **string**, **boolean**) are the **primitive** types you should use. Capitalized (**Number**, **String**) are the **wrapper object** types — almost never what you want, and not assignable to the primitives. **Hook:** *always lowercase.*

```
let a: string = 'hi'; // correct
let b: String = 'hi'; // object wrapper – avoid
// const c: string = b; // error
```

LEVEL 2 — Generics, Narrowing & OOP

Where real TypeScript skill shows. Expect deep follow-ups here.

Q27. What are generics and why use them?

Generics let you write code that works over **many types while preserving the relationship** between them. Instead of **any** (which loses type info), **<T>** flows the actual type through. **Hook:** *a type variable, like a function argument but for types.*

```
function identity<T>(x: T): T { return x; }
const a = identity('hi'); // T = string -> returns string
```

Q28. How do generic functions infer their type?

TS usually **infers** the type argument from the call, so you rarely pass it explicitly. You can pass it manually when inference isn't enough.

```
function first<T>(arr: T[]): T | undefined { return arr[0]; }
first([1, 2, 3]);           // T inferred as number
first<string>([]);          // explicit
```

Q29. What are generic constraints (**extends**)?

Restrict a type parameter so it must have certain members. **T extends { length: number }** means "T must have a length". This lets you safely access those members inside.

```
function longest<T extends { length: number }>(a: T, b: T): T {
  return a.length >= b.length ? a : b;
}
```

Q30. Generic interfaces and classes?

Interfaces and classes can take type parameters too, parameterizing their members.

```
interface Box<T> { value: T; }
class Stack<T> { private items: T[] = []; push(x: T) { this.items.push(x); } }
```

Q31. Default generic parameters?

Provide a fallback type so callers can omit the argument.

```
interface ApiResponse<T = unknown> { data: T; status: number; }
let r: ApiResponse = { data: {}, status: 200 }; // T = unknown
```

Q32. What does the **keyof** operator do?

Produces a **union of the keys** of a type as literal types. Foundational for type-safe property access.

```
interface User { id: number; name: string; }
type K = keyof User; // 'id' | 'name'
```

Q33. What is the `typeof` type operator?

In a `*type*` position, `typeof x` gives the **type of a value/variable**. Combine with `keyof` to derive types from runtime objects.

```
const cfg = { host: 'localhost', port: 5432 };
type Cfg = typeof cfg;           // { host: string; port: number }
type CfgKey = keyof typeof cfg;  // 'host' | 'port'
```

Q34. What are indexed access types (`T[K]`)?

Look up the type of a property by key. Combine with `keyof` for fully generic, type-safe getters.

```
interface User { id: number; name: string; }
type IdType = User['id'];           // number
function get<T, K extends keyof T>(o: T, k: K): T[K] { return o[k]; }
```

Q35. What is type narrowing?

TS uses **control-flow analysis** to refine a broad type to a more specific one inside a branch. The tools are `typeof`, `instanceof`, `in`, truthiness, equality, and custom guards. **Hook:** `*narrowing = teaching TS what you already proved with an if.*`

Q36. `typeof` type guards?

Use the runtime `typeof` operator in conditions to narrow primitives.

```
function f(x: string | number) {
  if (typeof x === 'string') x.toUpperCase();
  else x.toFixed(2);
}
```

Q37. `instanceof` type guards?

Narrow to a class instance using `instanceof` — works on anything with a constructor/prototype.

```
function f(x: Date | string) {
  if (x instanceof Date) return x.getFullYear();
  return x.length;
}
```

Q38. `in` operator narrowing?

Check whether a property **exists** on an object to discriminate between shapes.

```
type Fish = { swim(): void }; type Bird = { fly(): void };
function move(a: Fish | Bird) {
  if ('swim' in a) a.swim(); else a.fly();
}
```

Q39. Truthiness & equality narrowing?

An `if (x)` removes falsy types (`null`, `undefined`, `''`, `0`...). Comparing to a literal also narrows on both sides.

```
function f(x?: string | null) {
  if (x) x.trim();           // x is string here
}
```


Q40. What are user-defined type guards (**is**)?

A function returning a **type predicate** `arg is T` tells TS that, when it returns true, `arg` is `T`. Reusable, named narrowing.

```
function isString(x: unknown): x is string {
  return typeof x === 'string';
}
if (isString(val)) val.toUpperCase();
```

Q41. What is a discriminated (tagged) union?

A union of object types that share a **common literal kind field**. Switching on that field narrows precisely. This is the idiomatic way to model states/variants in TS.

```
type Shape =
  | { kind: 'circle'; r: number }
  | { kind: 'square'; size: number };
function area(s: Shape) {
  switch (s.kind) {
    case 'circle': return Math.PI * s.r ** 2;
    case 'square': return s.size ** 2;
  }
}
```

Q42. How do you do exhaustiveness checking?

Add a **default** branch that assigns the value to **never**. If you later add a new union member and forget to handle it, the assignment **fails to compile** — a compile-time safety net.

```
function area(s: Shape) {
  switch (s.kind) {
    case 'circle': return 0;
    case 'square': return 0;
    default: const _x: never = s; return _x;
  }
}
```

Q43. What are function overloads?

Declare **multiple call signatures** for one function, then a single implementation. Use when return type depends on argument shape in ways a union can't express cleanly.

```
function len(x: string): number;
function len(x: any[]): number;
function len(x: string | any[]): number { return x.length; }
```

Q44. How do you type **this** in functions?

Declare a fake first parameter named **this** (erased at runtime) to type the calling context.

```
interface Btn { label: string; }
function onClick(this: Btn) { console.log(this.label); }
```

Q45. Class access modifiers: public / private / protected?

public (default) = accessible everywhere. **private** = only inside the same class. **protected** = the class **and subclasses**. These are **compile-time** only (use **#field** for true runtime privacy).

```
class Account {
  public id = 1;
  private pin = '0000';
  protected balance = 0;
}
```

Q46. What are parameter properties?

A shorthand: add an access modifier to a constructor parameter and TS auto-declares **and** assigns the field. Cuts boilerplate dramatically.

```
class User {
  constructor(public id: number, private name: string) {}
} // creates this.id and this.name automatically
```

Q47. What are abstract classes?

A base class you **can't instantiate** directly; it can declare **abstract** members that subclasses **must** implement. Use for shared logic + enforced contract.

```
abstract class Animal {
  abstract sound(): string;
  describe() { return 'Says ' + this.sound(); }
}
```

Q48. Classes implementing interfaces?

implements enforces that a class satisfies an interface's shape. A class can implement many interfaces. It checks the contract but doesn't inherit code.

```
interface Logger { log(m: string): void; }
class ConsoleLogger implements Logger {
  log(m: string) { console.log(m); }
}
```

Q49. **private** (TS) vs **#private** (JS) fields?

TS **private** is enforced only by the compiler — at runtime the field is still accessible. JS **#name** is **truly private** at runtime (a hard error to access from outside). Prefer **#** when real encapsulation matters.

```
class A { private a = 1; #b = 2; } // a is reachable at runtime, #b is not
```

Q50. What are index signatures?

Describe objects with **dynamic, unknown keys** of a uniform value type. **Record<K, V>** is often a cleaner equivalent.

```
interface Scores { [name: string]: number; }
const s: Scores = { suraj: 90, alex: 85 };
```

Q51. What are utility types? Name the core ones.

Built-in generic type transformers. Core set: **Partial**, **Required**, **Readonly**, **Pick**, **Omit**, **Record**, **Exclude**, **Extract**, **NonNullable**, **ReturnType**, **Parameters**, **Awaited**. **Hook**: *prebuilt type math so you don't repeat yourself.*

Q52. **Partial**, **Required**, **Readonly**?

Partial<T> makes all props optional (great for updates/patches). **Required**<T> makes all required. **Readonly**<T> makes all readonly.

```
interface User { id: number; name?: string; }  
type Patch = Partial<User>;           // all optional  
type Full  = Required<User>;          // all required
```

Q53. **extends** for constraints vs **extends** in conditional types?

Same keyword, two jobs. In **<T extends X>** it's a **constraint** (T must be assignable to X). In **T extends X ? A : B** it's a **conditional test** (an if at the type level). Knowing they're different uses of one keyword is a common interview clarifier.

```
function f<T extends object>(x: T) {}           // constraint  
type IsArr<T> = T extends any[] ? true : false; // condition
```

LEVEL 3 — Advanced Type Manipulation

Type-level programming: utility, mapped, conditional & template types.

Q54. **Pick** and **Omit**?

Pick<T, K> keeps only the listed keys. **Omit**<T, K> removes the listed keys. They are opposites.

Hook: *Pick = whitelist, Omit = blacklist.*

```
interface User { id: number; name: string; pwd: string; }
type Public = Omit<User, 'pwd'>;    // { id, name }
type Cred   = Pick<User, 'name' | 'pwd'>;
```

Q55. What is **Record**<K, V>?

Builds an object type with keys **K** and values of type **V**. The clean way to type maps/dictionaries with known or unioned keys.

```
type Roles = Record<'admin' | 'user', boolean>;
// { admin: boolean; user: boolean }
```

Q56. **Exclude** and **Extract**?

Operate on **unions**. **Exclude**<T, U> removes members of **U** from **T**. **Extract**<T, U> keeps only the members assignable to **U**. (Compare to Pick/Omit which work on object keys.)

```
type T = 'a' | 'b' | 'c';
type X = Exclude<T, 'a'>;    // 'b' | 'c'
type Y = Extract<T, 'a' | 'z'>; // 'a'
```

Q57. What is **NonNullable**<T>?

Removes **null** and **undefined** from a type.

```
type T = string | null | undefined;
type Clean = NonNullable<T>;    // string
```

Q58. **ReturnType** and **Parameters**?

Extract a function's **return type** or its **parameter tuple** from its type. Hugely useful for deriving types from existing functions instead of duplicating.

```
function make() { return { id: 1, ok: true }; }
type R = ReturnType<typeof make>;    // { id: number; ok: boolean }
type P = Parameters<(a: string, b: number) => void>; // [string, number]
```

Q59. What does **Awaited**<T> do?

Recursively unwraps the value type of a **Promise** (handles nested promises). Mirrors what **await** produces.

```
type A = Awaited<Promise<string>>;    // string
type B = Awaited<Promise<Promise<number>>>>; // number
```

Q60. InstanceType and ConstructorParameters?

`InstanceType<C>` = the instance type a constructor produces. `ConstructorParameters<C>` = its constructor args as a tuple.

```
class Service { constructor(public url: string) {} }
type I = InstanceType<typeof Service>; // Service
type C = ConstructorParameters<typeof Service>; // [string]
```

Q61. What are mapped types?

Generate a new object type by **iterating over the keys** of another with `[K in keyof T]`. This is how `Partial`, `Readonly`, etc. are built internally.

```
type Optional<T> = { [K in keyof T]?: T[K] };
type Stringify<T> = { [K in keyof T]: string };
```

Q62. What are mapping modifiers?

Inside a mapped type you can **add or remove** `readonly` and `?` using `+/-` prefixes. `-?` makes everything required; `-readonly` makes everything mutable.

```
type Mutable<T> = { -readonly [K in keyof T]: T[K] };
type Concrete<T> = { [K in keyof T]-?: T[K] };
```

Q63. What is key remapping with `as`?

Rename or **filter** keys while mapping, using `as`. Combine with template literals to build things like getter names; map a key to `never` to drop it.

```
type Getters<T> = {
  [K in keyof T as `get${Capitalize<string & K>}`]: () => T[K]
};
```

Q64. What are conditional types?

A type-level `if: T extends U ? X : Y. Picks one type based on assignability. The engine behind most utility types.`

```
type IsString<T> = T extends string ? 'yes' : 'no';
type A = IsString<'hi'>; // 'yes'
type B = IsString<42>; // 'no'
```

Q65. What does the `infer` keyword do?

Inside a conditional type's `extends` clause, `infer R` **captures** a piece of a type into a variable you can use in the true branch. This is how `ReturnType`, `Awaited`, `element-of`, etc. work.

```
type ElementOf<T> = T extends (infer U)[] ? U : never;
type E = ElementOf<number[]>; // number
type Ret<F> = F extends (...a: any) => infer R ? R : never;
```

Q66. What are distributive conditional types?

When the checked type is a **naked type parameter** that's a union, the conditional **distributes** over each member. Wrap both sides in `[...]` to disable distribution.

```
type ToArray<T> = T extends any ? T[] : never;
type A = ToArray<string | number>; // string[] | number[]
type NoDist<T> = [T] extends [any] ? T[] : never; // (string|number)[]
```

Q67. What are template literal types?

String literal types built with template syntax. They can interpolate unions to generate combinations — great for typed event names, routes, CSS units.

```
type Event = 'click' | 'hover';
type Handler = `on${Capitalize<Event>}`; // 'onClick' | 'onHover'
```

Q68. Intrinsic string manipulation types?

Built-ins for the type level: `Uppercase`, `Lowercase`, `Capitalize`, `Uncapitalize`. Pair them with template literal types.

```
type A = Uppercase<'abc'>; // 'ABC'
type B = Capitalize<'name'>; // 'Name'
```

Q69. What are recursive types?

A type that refers to itself — used for trees, JSON, linked lists, and deep transformations like `DeepReadonly`.

```
type Json = string | number | boolean | null | Json[] | { [k: string]: Json };
type DeepReadonly<T> = { readonly [K in keyof T]: DeepReadonly<T[K]> };
```

Q70. What does the `satisfies` operator do? (TS 4.9)

Checks that a value **conforms to** a type **without widening** its inferred type. You keep the precise literal type `*and*` get validation. Solves the old "annotate vs infer" tradeoff.

```
const cfg = {
  host: 'localhost', port: 5432
} satisfies Record<string, string | number>;
cfg.port.toFixed(); // port still known as number, not string|number
```

Q71. What are `const` type parameters? (TS 5.0)

Prefix a type parameter with `const` so arguments are inferred as their **narrow, literal** types — like an automatic `as const` at the call site.

```
function pick<const T>(x: T): T { return x; }
const r = pick(['a', 'b']); // readonly ['a', 'b'], not string[]
```

Q72. Explain variance (covariance / contravariance).

Describes how subtype relationships flow through containers. Return types/positions are **covariant** (a more specific type is fine). Function **parameters** are **contravariant** under `strictFunctionTypes`.

Hook: `*outputs narrow safely, inputs widen safely.*`

Q73. How do you create nominal / branded types?

TS is structural, so two `string` IDs are interchangeable. Add a phantom **brand** property to make them incompatible, preventing mix-ups (e.g. `UserId` vs `OrderId`).

```
type UserId = string & { readonly __brand: 'UserId' };
const id = 'u1' as UserId;
// a plain string is no longer assignable to UserId
```

Q74. What is `strictFunctionTypes`?

Enables stricter, **contravariant** checking of function parameter types (except on methods, for compatibility). Catches unsound callback assignments. Part of `strict`.

Q75. What is declaration merging?

TS merges multiple declarations with the **same name** into one. Two `interfaces` with the same name combine their members; namespaces and interfaces can also merge. (Types/`type` aliases cannot merge.)

```
interface Box { w: number; }
interface Box { h: number; }
// Box is now { w: number; h: number }
```

Q76. What is module augmentation?

Use `declare module '...'` (or global augmentation) to **add types to existing modules** you don't own — e.g. extending Express's `Request` or a library's interface.

```
declare module 'express' {
  interface Request { userId?: string; }
}
```

Q77. What are ambient declarations / `declare`?

`declare` says "this exists at runtime, here's its type" without emitting code. Used to describe globals or JS libraries that have no types.

```
declare const VERSION: string;
declare function gtag(...args: any[]): void;
```

Q78. What are `.d.ts` declaration files?

Type-only files that describe the shape of JS code (no implementation). `tsc` can emit them for your library so consumers get types; `@types/*` packages ship them for untyped libraries.

Q79. Namespaces vs ES modules?

Namespaces (`namespace X {}`) are a legacy TS way to group code in one global scope. **ES modules** (`import/export`) are the modern standard — prefer modules; reserve namespaces mainly for organizing ambient `.d.ts` types.

Q80. What are type-only imports/exports?

`import type` / `export type` import only the **type**, guaranteeing it's erased and never causes a runtime import or side effect. Required with `isolatedModules/verbatimModuleSyntax` and helps bundlers tree-shake.

```
import type { User } from './types';
import { type Role, createRole } from './roles';
```

Q81. What are variadic tuple types?

Tuples can use a spread `...T` of another tuple/array type, so you can build and concatenate tuple types generically. This is how strongly-typed `concat`, `curry`, and `Parameters` manipulation work.

```
type Concat<A extends any[], B extends any[]> = [...A, ...B];
type R = Concat<[1, 2], [3, 4]>; // [1, 2, 3, 4]
```

Q82. What is `NoInfer<T>`? (TS 5.4)

A built-in that **blocks** a type parameter from being inferred at a given position, forcing TS to infer it from somewhere else. Useful when one argument should drive inference and another should only be validated against it.

```
function pick<T>(items: T[], def: NoInfer<T>): T { return def; }
pick(['a', 'b'], 'a'); // T inferred from items, not def
```


LEVEL 4 — Config, Tooling & Real-World Gotchas

Senior-level questions: how TS actually compiles, and the traps.

Q83. Which `tsconfig.json` options matter most?

`strict` (umbrella for safety flags), `target` (JS output version), `module/moduleResolution`, `outDir/rootDir`, `lib`, `esModuleInterop`, `skipLibCheck`, `paths` (aliases), and `noEmit` (type-check only, common when a bundler does the build).

Q84. What does `strict: true` enable?

A bundle of strict flags: `noImplicitAny`, `strictNullChecks`, `strictFunctionTypes`, `strictBindCallApply`, `strictPropertyInitialization`, `noImplicitThis`, `alwaysStrict`, and `useUnknownInCatchVariables`. Always start new projects with it on.

Q85. What does `noImplicitAny` do?

Errors when TS would otherwise silently infer `any` (e.g. an un-annotated parameter). Forces you to be explicit and prevents accidental loss of type safety.

```
function f(x) {} // error under noImplicitAny: x implicitly any
```

Q86. How does TypeScript compile / what is type erasure?

`tsc` (or `esbuild/swc/babel`) type-checks, then **strips all type annotations** and emits plain JS. **Types do not exist at runtime** — you can't `if (x is MyType)` at runtime. Exceptions that leave runtime artifacts: `enum`, classes, and (with metadata) decorators.

Q87. What's the deal with decorators?

Functions that annotate/transform classes and members with `@`. **Stage 3** decorators are supported natively in **TS 5.0+**. The older style needs `experimentalDecorators` (used by Angular, NestJS, TypeORM). The two are not identical — know which your framework uses.

```
function logged(target: any, ctx: ClassMethodDecoratorContext) {}
class S { @logged run() {} }
```

Q88. Why do many teams avoid `enum`?

Numeric enums create a runtime object and a confusing reverse map; `const enum` breaks under `isolatedModules`. A **union of string literals** or an `as const` object is lighter, fully erased, and easier to refactor.

```
const Status = { Active: 'active', Done: 'done' } as const;
type Status = typeof Status[keyof typeof Status];
```

Q89. Index signature vs `Record` vs `Map`?

Index signature / `Record` = a plain object with typed keys/values, keys are strings/numbers/symbols, serializes to JSON. `Map` = a real runtime structure with **any** key type, insertion order, `.size`, and better for frequent add/delete. Choose `Map` for dynamic collections, objects for fixed-shape data.

Q90. Why is the `catch` variable `unknown`?

With `useUnknownInCatchVariables` (in `strict`), the caught error is typed `unknown` because JS can throw **anything**, not just `Error`. You must narrow before using it — safer than the old implicit `any`.

```
try { /* ... */ }
catch (e) {
  if (e instanceof Error) console.log(e.message);
}
```

Q91. What is the array covariance gotcha?

TS arrays are **covariant**, which is technically unsound: `Dog[]` is accepted where `Animal[]` is expected, so you could **push** a `Cat` into it. TS allows this for ergonomics — be aware when mutating shared arrays.

Q92. What are excess property checks?

When you assign an **object literal** directly, TS flags properties not in the target type — catching typos. Assigning through a variable skips this check (structural rules apply instead).

```
interface Opts { width: number; }
const o: Opts = { width: 5, height: 9 }; // error: 'height' unknown
const tmp = { width: 5, height: 9 }; const o2: Opts = tmp; // ok
```

Q93. Why avoid the bare `Function` type?

`Function` accepts any callable but gives **no parameter or return safety** (returns `any`). Always use a specific signature like `(x: number) => void`.

```
let bad: Function; // unsafe
let good: (x: number) => string;
```

Q94. How do you type async functions and Promises?

An **async** function always returns a `Promise`. Annotate the **resolved** type as `Promise<T>`; `await` unwraps it. Reach for `unknown` then narrow when the payload isn't trusted.

```
async function getUser(id: number): Promise<User> {
  const res = await fetch('/u/' + id);
  return res.json() as Promise<User>;
}
```

Q95. `Omit`/`Pick` vs `Exclude`/`Extract` — what trips people up?

`Pick`/`Omit` operate on the **keys of an object type**. `Exclude`/`Extract` operate on the **members of a union**. Mixing them up is a classic interview trap. **Hook:** `*objects -> Pick/Omit; unions -> Extract/Exclude.*`

Q96. `@ts-ignore` vs `@ts-expect-error`?

Both suppress an error on the next line. `@ts-expect-error` is better: if the line **stops erroring**, TS flags the now-unnecessary comment — so it self-cleans as you fix code. Use it sparingly and with a reason.

```
// @ts-expect-error legacy lib has wrong types
legacy.doThing(badArg);
```

Q97. Practical tips for migrating JavaScript to TypeScript?

Rename `.js` -> `.ts` incrementally; enable `allowJs` and `checkJs` to type-check JS gradually; start with loose settings then tighten to `strict`; install `@types/*` for libraries; replace `any` with `unknown` + narrowing; and treat the compiler's errors as a prioritized to-do list rather than noise.

Q98. What do `esModuleInterop` and `skipLibCheck` do?

`esModuleInterop` smooths importing CommonJS modules with default-import syntax (`import express from 'express'`) by adding interop helpers. `skipLibCheck` skips type-checking of `.d.ts` files — speeds up builds and avoids errors from conflicting third-party types.

Q99. `tsc` vs bundlers (`esbuild/swc`) — and `isolatedModules`?

`tsc` both type-checks and emits JS. Fast tools like `esbuild/swc/Babel` transpile each file in **isolation** (no cross-file type info) for speed, so you run `tsc --noEmit` separately for checking. `isolatedModules` makes TS warn about patterns those single-file transpilers can't handle (e.g. re-exporting a type without `export type`).

Q100. How do you share types between a frontend and backend?

Define types **once** and import them in both — via a shared package/workspace, a `types` folder, or by deriving client types from server schemas (Zod, tRPC, OpenAPI codegen). Single source of truth means an API change surfaces as a **compile error** on the client instead of a runtime bug.

```
// shared/types.ts
export interface Event { id: string; title: string; }
// imported by both Next.js client and Express server
```